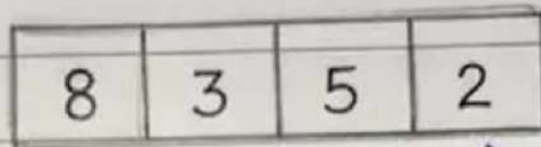
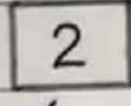
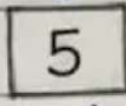
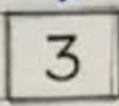
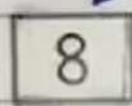


HOW MERGE SORT WORKS?

Initial List
(unsorted)



Divide

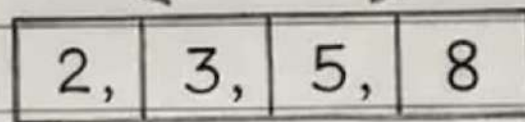


Sorted
Sub-lists

Merge

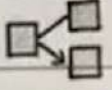
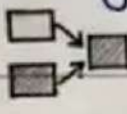


Merge



Sorted Output

Working Flow

1.  Input list is recursively divided in half.
2. Halves are split until single elements remain.
3. Single elements are considered sorted lists.
4.  Pairs of sorted lists are merged into larger sorted lists.

The screenshot displays a code editor with a C program implementing Merge Sort. The program is named `MergeQ1.c` and is being edited in a window titled `C MergeQ1.c U`. The code is as follows:

```
1 //A school has recently conducted a scholarship examination for all students of Class 12. The examination
2 department has collected the marks of all students in an array.
3
4 #include <stdio.h>
5 void merge(int arr[], int l, int m, int r) {
6
7     int i, j, k;
8     int n1 = m - l + 1;
9     int n2 = r - m;
10    int L[n1], R[n2];
11    for (i = 0; i < n1; i++)
12        L[i] = arr[l + i];
13    for (j = 0; j < n2; j++)
14        R[j] = arr[m + 1 + j];
15    i = 0;
16    j = 0;
17    k = l;
18    while (i < n1 && j < n2) {
19        if (L[i] <= R[j]) {
20            arr[k] = L[i];
21            i++;
22        } else {
23            arr[k] = R[j];
24            j++;
25        }
26        k++;
27    }
28    while (i < n1) {
29        arr[k] = L[i];
30        i++;
31        k++;
32    }
33    while (j < n2) {
34        arr[k] = R[j];
35        j++;
36        k++;
37    }
38 }
39
40 void mergeSort(int arr[], int l, int r) {
41     if (l < r) {
42         int m = l + (r - l) / 2;
43         mergeSort(arr, l, m);
44         mergeSort(arr, m + 1, r);
45         merge(arr, l, m, r);
46     }
47 }
48
49 int main() {
50     int N, i;
51     printf("Enter the number of students: ");
52     scanf("%d", &N);
53     int arr[N];
54     printf("Enter the marks of %d students:\n", N);
55     for (i = 0; i < N; i++) {
56         scanf("%d", &arr[i]);
57     }
58     mergeSort(arr, 0, N - 1);
59
60     printf("Sorted marks in ascending order:\n");
61     for (i = 0; i < N; i++) {
62         printf("%d ", arr[i]);
63     }
64     printf("\n");
65     return 0;
66 }
```

The output of the program is shown in the terminal window on the right, titled `CHAT TERMINAL`. The output is as follows:

```
abhinavsharma@Abhinavs-MacBook-Air C % cd "/Users/abhinavsharma/Comp. Language/C/" && gcc MergeQ1.c -o MergeQ1 && ./MergeQ1
Enter the number of students: 5
Enter the marks of 5 students:
54
76
80
43
65
Sorted marks in ascending order:
43 54 65 76 89
abhinavsharma@Abhinavs-MacBook-Air C %
```

```
C MergeQ2.c x C MergeQ2.c x C MergeQ2.c x C MergeQ2.c x C MergeQ2.c x C MergeQ2.c x
1 /*An e-commerce company stores product prices in the order they are added to the platform. Before
2 publishing a seasonal sales report, the company wants all product prices to be arranged in ascending order.*/
3 #include <stdio.h>
4 void merge(int arr[], int l, int m, int r) {
5
6     int i, j, k;
7     int n1 = m - l + 1;
8     int n2 = r - m;
9
10    int L[n1], R[n2];
11    for (i = 0; i < n1; i++)
12        L[i] = arr[l + i];
13    for (j = 0; j < n2; j++)
14        R[j] = arr[m + 1 + j];
15
16    i = 0;
17    j = 0;
18    k = l;
19    while (i < n1 && j < n2) {
20        if (L[i] <= R[j]) {
21            arr[k] = L[i];
22            i++;
23        } else {
24            arr[k] = R[j];
25            j++;
26        }
27        k++;
28    }
29    while (i < n1) {
30        arr[k] = L[i];
31        i++;
32        k++;
33    }
34    while (j < n2) {
35        arr[k] = R[j];
36        j++;
37        k++;
38    }
39 }
40
41 void mergeSort(int arr[], int l, int r) {
42     if (l < r) {
43         int m = l + (r - l) / 2;
44         mergeSort(arr, l, m);
45         mergeSort(arr, m + 1, r);
46         merge(arr, l, m, r);
47     }
48 }
49
50 int main() {
51     int n, i;
52     printf("Enter the number of products: ");
53     scanf("%d", &n);
54     int arr[n];
55     printf("Enter the prices of %d products:\n", n);
56     for (i = 0; i < n; i++) {
57         scanf("%d", &arr[i]);
58     }
59
60     mergeSort(arr, 0, n - 1);
61
62     printf("Sorted prices in ascending order:\n");
63     for (i = 0; i < n; i++) {
64         printf("%d ", arr[i]);
65     }
66     printf("\n");
67     return 0;
68 }
```

```
abhinavsharma@Abhinavs-MacBook-Air C % cd "/Users/abhinavsharma/Comp. Language/C/" && gcc MergeQ2.c -o MergeQ2 && "/Users/abhinavsharma/Comp. Language/C/" MergeQ2
Enter the number of products: 5
Enter the prices of 5 products:
100
564
564
33
234
Sorted prices in ascending order:
33 100 234 564 564
abhinavsharma@Abhinavs-MacBook-Air C %
```

```
C MergeQ3.c u C MergeQ3.c u C MergeQ3.c u C MergeQ3.c u C MergeQ3.c u X
1 //A multinational company has employee salary records stored in an array. Before
2 //generating annual reports,
3 //the human Resources department wants the salary records arranged in ascending or
4 //descending order.
5 #include <stdio.h>
6
7 void mergeSort(int arr[], int l, int m, int r) {
8     int i, j, k;
9     int n1 = m - l + 1;
10    int n2 = r - m;
11
12    int L[n1], R[n2];
13
14    for (i = l; i < m; i++)
15        L[i] = arr[i + l];
16    for (j = m; j < r; j++)
17        R[j] = arr[m + 1 + j];
18
19    i = 0;
20    j = 0;
21    k = l;
22    while (i < n1 && j < n2) {
23        if (L[i] <= R[j]) {
24            arr[k] = L[i];
25            i++;
26        } else {
27            arr[k] = R[j];
28            j++;
29        }
30        k++;
31    }
32    while (i < n1) {
33        arr[k] = L[i];
34        i++;
35        k++;
36    }
37    while (j < n2) {
38        arr[k] = R[j];
39        j++;
40        k++;
41    }
42 }
43
44 void mergeSort(int arr[], int l, int r) {
45     if (l < r) {
46         int m = l + (r - l) / 2;
47         mergeSort(arr, l, m);
48         mergeSort(arr, m + 1, r);
49         merge(arr, l, m, r);
50     }
51 }
52
53 int main() {
54     int n, i;
55     printf("Enter the number of employees: ");
56     scanf("%d", &n);
57     if (n <= 0) {
58         printf("No employees to sort.\n");
59         return 0;
60     }
61
62     int arr[n];
63     printf("Enter the salaries of %d employees:\n", n);
64     for (i = 0; i < n; i++) {
65         scanf("%d", &arr[i]);
66     }
67     mergeSort(arr, 0, n - 1);
68
69     for (i = 0; i < n; i++) {
70         printf("%d ", arr[i]);
71     }
72     printf("\n\nLowest salary: %d\n", arr[0]);
73     printf("\n\nHighest salary: %d\n", arr[n - 1]);
74     printf("\n\nDifference : %d\n", arr[n - 1] - arr[0]);
75     return 0;
76 }
77
```

```
abhinavsharma@Abhinavs-MacBook-Air C % cd "/Users/abhinavsharma/Comp. Language/C/" && gcc MergeQ3.c -o MergeQ3 && ./MergeQ3
Enter the number of employees: 5
Enter the salaries of 5 employees:
5000
50000
75000
89000
34000
5000 34000 50000 75000 89000
Lowest salary: 5000
Highest salary: 89000
Difference : 84000
abhinavsharma@Abhinavs-MacBook-Air C %
```


HOW QUICK SORT WORKS ?

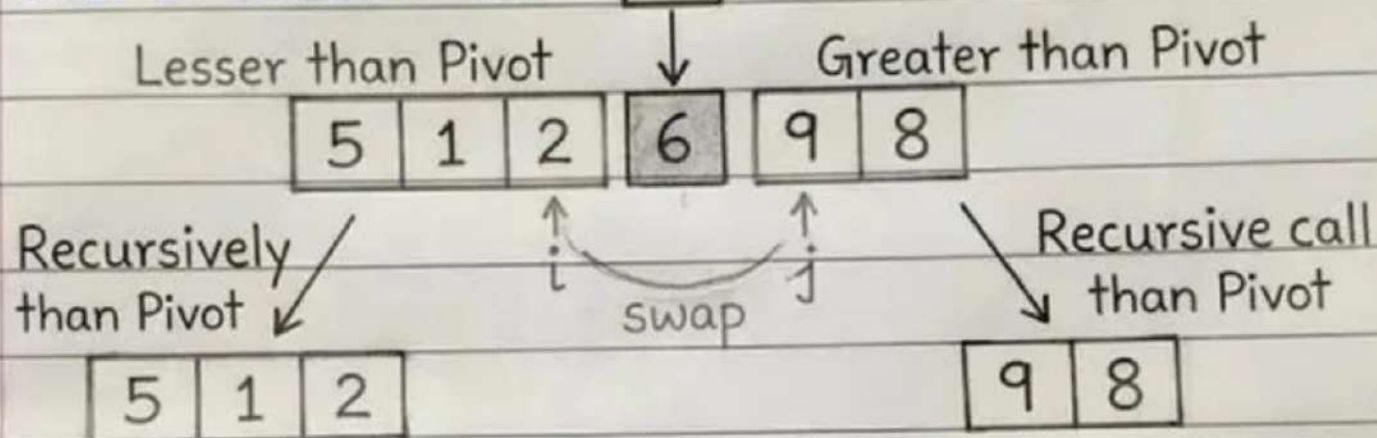
5	9	1	6	2	8
---	---	---	---	---	---

 ← Input Unsorted Array

STEP 1: Choose Pivot →

6

 PIVOT



1	2	5	6	8	9
---	---	---	---	---	---

 Fully Sorted Output

Sorting Flow

1. Choose a 'pivot' element. 

6

2. Partition the array around the pivot. $\frac{5}{8}$
3. Place smaller numbers to the left. $\Rightarrow$
4. Place larger numbers to the right. $\Rightarrow$
5. Recursively repeat for partitions. ⚡

```
C QuickQ1.c U X C MergeQ2.c U C MergeQ1.c U C Quick2.c U C QuickQ3.c U C MergeQ3
1 /*A sports academy records the completion times (in seconds) of runners participating in a race. The coach
2 wants to arrange the timings from the fastest runner to the slowest runner.
3 */
4 #include <stdio.h>
5 int partition(int arr[], int low, int high) {
6     int pivot = arr[high]; // pivot
7     int i = (low - 1); // Index of smaller element
8
9     for (int j = low; j <= high - 1; j++) {
10         if (arr[j] <= pivot) {
11             i++;
12             int temp = arr[i];
13             arr[i] = arr[j];
14             arr[j] = temp;
15         }
16     }
17     int temp = arr[i + 1];
18     arr[i + 1] = arr[high];
19     arr[high] = temp;
20     return (i + 1);
21 }
22 void quickSort(int arr[], int low, int high) {
23     if (low < high) {
24         int pi = partition(arr, low, high);
25         quickSort(arr, low, pi - 1);
26         quickSort(arr, pi + 1, high);
27     }
28 }
29
30 void printArray(int arr[], int size) {
31     for (int i = 0; i < size; i++)
32         printf("%d ", arr[i]);
33     printf("\n");
34 }
35
36 int main() {
37     int n;
38     printf("Enter the number of runners: ");
39     scanf("%d", &n);
40     int arr[n];
41     for (int i = 0; i < n; i++) {
42         printf("Enter the completion time for runner %d: ", i + 1);
43         scanf("%d", &arr[i]);
44     }
45     quickSort(arr, 0, n - 1);
46
47     printArray(arr, n);
48     return 0;
49 }
```

```
abhinavsharma@Abhinavs-MacBook-Air C % cd "/Users/abhinavs
/" && gcc QuickQ1.c -o QuickQ1 && "/Users/abhinavsharma/Com
1
Enter the completion time for runner 1: 78
Enter the completion time for runner 2: 35
Enter the completion time for runner 3: 75
Enter the completion time for runner 4: 59
Enter the completion time for runner 5: 39
35 39 59 75 78
abhinavsharma@Abhinavs-MacBook-Air C %
```

Ln 34, Col 18 Spaces: 4 UTF-8 LF C 9-9 Go Live macos-clang-arm64 Prettier

mergeQ2.c U

C MergeQ1.c U

C Quickq2.c U X

C QuickQ3.c U

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

C Quickq2.c > main()

/*A library stores books using unique identification numbers. Due to years

are no longer arranged properly.*/

#include <stdio.h>

int partition(int arr[], int low, int high) {

int pivot = arr[high];

int i = (low - 1);

for (int j = low; j <= high - 1; j++) {

if (arr[j] <= pivot) {

i++;

int temp = arr[i];

arr[i] = arr[j];

arr[j] = temp;

}

int temp = arr[i + 1];

arr[i + 1] = arr[high];

arr[high] = temp;

return (i + 1);

}

void quickSort(int arr[], int low, int high) {

if (low < high) {

int pi = partition(arr, low, high);

quickSort(arr, low, pi - 1);

quickSort(arr, pi + 1, high);

}

}

void printArray(int arr[], int size) {

for (int i = 0; i < size; i++)

printf("%d ", arr[i]);

printf("\n");

}

int main() {

int n;

printf("Enter the number of books: ");

scanf("%d", &n);

int arr[n];

printf("Enter the IDs of %d books:\n", n);

for (int i = 0; i < n; i++) {

scanf("%d", &arr[i]);

}

quickSort(arr, 0, n - 1);

printArray(arr, n);

return 0;

}

CHAT

TERMINAL

abhinavsharma@Abhinavs-MacBook-Air C % cd "/Users/abhinavsharma/Comp. Language/C/" && gcc Quickq2.c -o Quickq2 && "/Users/abhinavsharma/Comp. Language/C/"Quickq2

Enter the number of books: 5

Enter the IDs of 5 books:

235

434

343

634

356

235 343 356 434 634

abhinavsharma@Abhinavs-MacBook-Air C %

main*

Ln 40, Col 38

Spaces: 4

UTF-8

LF

C

Go Live

macos-clang-arm64

Prettier

U

C MergeQ2.c U

C MergeQ1.c U

C Quickq2.c U

C QuickQ3.c U X

C MergeQ3.c U

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

C QuickQ3.c > main()

1 /*An online marketplace tracks the daily sales made by different sellers. At the end of each week

2

3

4 #include <stdio.h>

5 int partition(int arr[], int low, int high) {

6 int pivot = arr[high]; // pivot

7 int i = (low - 1); // Index of smaller element

8

9 for (int j = low; j <= high - 1; j++) {

10 if (arr[j] <= pivot) {

11 i++;

12 int temp = arr[i];

13 arr[i] = arr[j];

14 arr[j] = temp;

15 }

16 }

17 int temp = arr[i + 1];

18 arr[i + 1] = arr[high];

19 arr[high] = temp;

20 return (i + 1);

21 }

22 void quickSort(int arr[], int low, int high) {

23 if (low < high) {

24

25 int pi = partition(arr, low, high);

26

27 quickSort(arr, low, pi - 1);

28 quickSort(arr, pi + 1, high);

29 }

30 }

31 void printArray(int arr[], int size) {

32 for (int i = 0; i < size; i++)

33 printf("%d ", arr[i]);

34 printf("\n");

35 }

36 int main() {

37 int n, sum=0;

38 printf("Enter the number of sellers: ");

39 scanf("%d", &n);

40 int arr[n];

41 printf("Enter the sales figures of %d sellers:\n", n);

42 for (int i = 0; i < n; i++) {

43 scanf("%d", &arr[i]);

44 sum += arr[i];

45 }

46 quickSort(arr, 0, n - 1);

47 printArray(arr, n);

48 printf("TOP 3 : %d %d %d", arr[0], arr[1], arr[2]);

49 printf("\n");

50 printf("Average: %d", sum/n);

51 return 0;

52 }

CHAT

TERMINAL

zsh

abhinavsharma@Abhinavs-MacBook-Air C % cd "/Users/abhinavsharma/Comp. Language/C/" && gcc QuickQ3.c -o QuickQ3 && "/Users/abhinavsharma/Comp. Language/C/"QuickQ3

Enter the number of sellers: 5

Enter the sales figures of 5 sellers:

334

3532

543

642

54

54 334 543 642 3532

TOP 3 : 54 334 543

Average: 1021.8

abhinavsharma@Abhinavs-MacBook-Air C %

Ln 49, Col 18

Spaces: 4

UTF-8

LF

C

Go Live

macos-clang-arm64

Prettier